# CSC3002
# C/C++ Programming

# Designing Class

**Prof. Dr. Ruizhi Chen， SDS**

**Office: TxC 717**

# Designing Classes

6.1 Representing points
6.2 Operator overloading
6.3 Rational numbers
6.4 Designing a token scanner class
6.5 Encapsulating programs as classes

# Data Types in C++

- Primitive types include but not limited to:

| | |
|---|---|
| **`int`** | This type is used to represent integers, which are whole numbers such as 17 or −53. |
| **`double`** | This type is used to represent numbers that include a decimal fraction, such as 3.14159265. |
| **`bool`** | This type represents a logical value (**`true`** or **`false`**). |
| **`char`** | This type represents a single ASCII character. |

- Collections
- Compound data types:
  - Enumerated types
  - Structures
  - Classes (e.g., C++ strings and streams)

# Enumerated types

- An enumerated type can be roughly considered as a subset of **`int`** with special names for each value in the subset:

```
enum Direction { NORTH, EAST, SOUTH, WEST };
```

- When the C++ compiler encounters this definition, it assigns values to the constant names by numbering them consecutively starting with 0. Thus, NORTH is assigned the value 0, EAST is 1, SOUTH is 2, and WEST is 3.
- You can assign values by yourself:

```
enum Coin {
     PENNY = 1,
     NICKEL = 5,
     DIME = 10,
     QUARTER = 25,
     HALF_DOLLAR = 50,
     DOLLAR = 100
};
```

# Structures

- All modern higher-level languages offer some facility for representing ***structures***, which are compound values in which the individual components are specified by name.
- Given that C++ is a superset of the older C language, it is still possible to define classic C structures, which are defined using the following syntactic form:

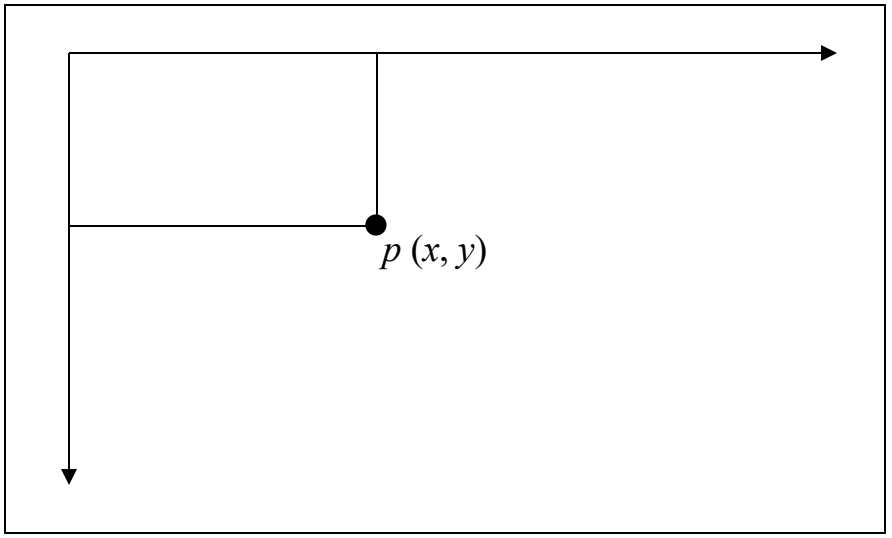
```
struct typename {
    declarations of fields
};
```

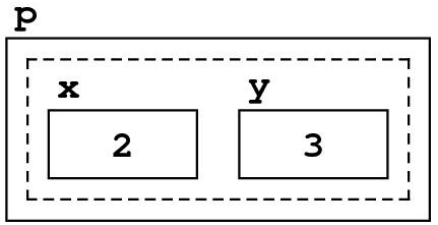
```
struct Point {
  int x;
  int y;
};
```

- This definition creates a *type,* not a *variable.* If you want to declare variables of the structure type, you use the type name just as you would with any other declaration.

# Representing Points

- One of the simplest examples of a data structure is a ***point***, which is composed of an *x* coordinate and a *y* coordinate component.
- C++ offers more than one model for representing a point value. The older style, which C++ inherits from C, is to defined the **`Point`** type as a ***structure***. The more modern approach is to define **`Point`** as a ***class***. The next several slides explore each of these models.



# Representing Points Using a Structure

- For most graphical devices, coordinates are measured in pixels on the screen, which means that these components *x* and *y* should be integers, since it is impossible to illuminate a fraction of a pixel.

```
struct Point {
   int x;
   int y;
};
```

- This definition allows you to declare a **`Point`** variable like this:

```
Point pt;
```

- Given the variable **`pt`**, you can select the individual ***fields*** or ***members*** using the dot operator (`.`), as in **`pt.x`** and **`pt.y`**.

# Representing Points Using a Structure

```
struct Point
{
    int x, y;

    Point():x(0),y(0){};
    Point(int xx, int yy):x(xx),y(yy){};

    Point &operator+(const Point pt);
    double length(const Point &p);
    double length();
    string toString();

    ~Point(){};
};
```

# Representing Points Using a Structure

```
Point &Point::operator+(const Point pt)
{
	this->x += pt.x;
	this->y += pt.y;
	return *this;
}
```

```
double Point::length(const Point &p)
{
	double dx = p.x - this->x;
	double dy = p.y - this->y;
	return sqrt(dx * dx + dy * dy);
}
```

```
double Point::length()
{
	double dx = this->x;
	double dy = this->y;
	return sqrt(dx * dx + dy * dy);
}
```

```
string Point::toString()
{
	ostringstream os;
	os << "(x,y,lenth)=(" << this->x << "," << this->y << "," << this-
	>length() << ")";
	return os.str();
}
```

# Representing Points Using a Class

```
struct Point
{
        int x, y;

        Point():x(0),y(0){};
        Point(int xx, int yy):x(xx),y(yy){};

        Point &operator+(const Point pt);
        double length(const Point &p);
        double length();
        string toString();

        ~Point(){};
};
```

**Structure for C++**

```
class Point
{
public:
        int x, y;

        Point() : x(0), y(0) {};
        Point(int xx, int yy) : x(xx), y(yy) {};

        Point &operator+(const Point pt);
        double length(const Point &p);
        double length();
        string toString();

        ~Point() {};
};
```

**Class**

# C++ struct vs. C struct

| Feature | C Struct | C++ Struct |
|---|---|---|
| **Default Access** | All members are `public` | All members are `public` (same as C) |
| **Functions Inside** | ❌ Cannot contain functions | ✅ Can contain member functions |
| **Constructors/Destruc** | ❌ Not supported | ✅ Fully supported |
| **Inheritance** | ❌ Not supported | ✅ Can inherit from other structs/classes |
| **Access Specifiers** | ❌ `public`/`private`/`protected` not available | ✅ Available (though rarely used with struct) |
| **Static Members** | ❌ Not supported | ✅ Supported |
| **Type Declaration** | Must use `struct` keyword when declaring variables | `struct` keyword optional when declaring |
| **Polymorphism** | ❌ Not supported | ✅ Supports virtual functions |

# Representing Points Using a Class

- Although structure types are part of the history of C++ and the languages that came before it, they have largely been supplanted by classes, which offer greater power and flexibility. The **`Point`** structure from the preceding section is identical to the following class definition:

```
class Point {
public:
   int x;
   int y;
};
```

- This definition allows you to declare a **`Point`** object like this:

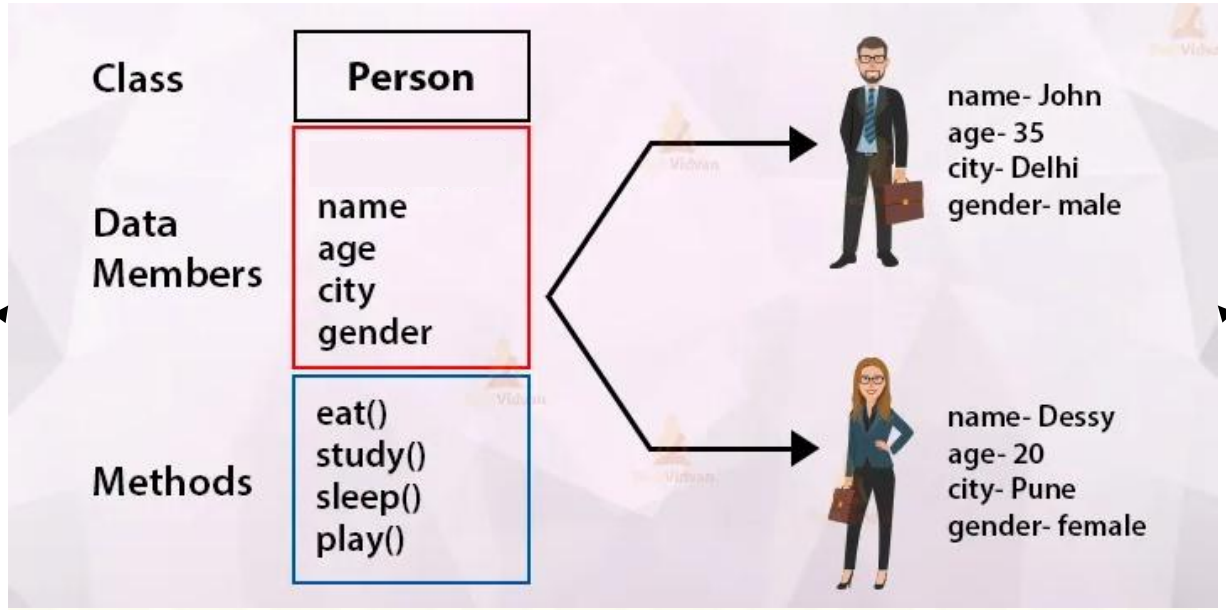
```
Point pt;
```

- As the x and y fields are included as part of the **`public`** section, which is visible to clients, you can select the public ***fields*** using the dot operator (**`.`**), as in **`pt.x`** and **`pt.y`**.

# Classes and Objects

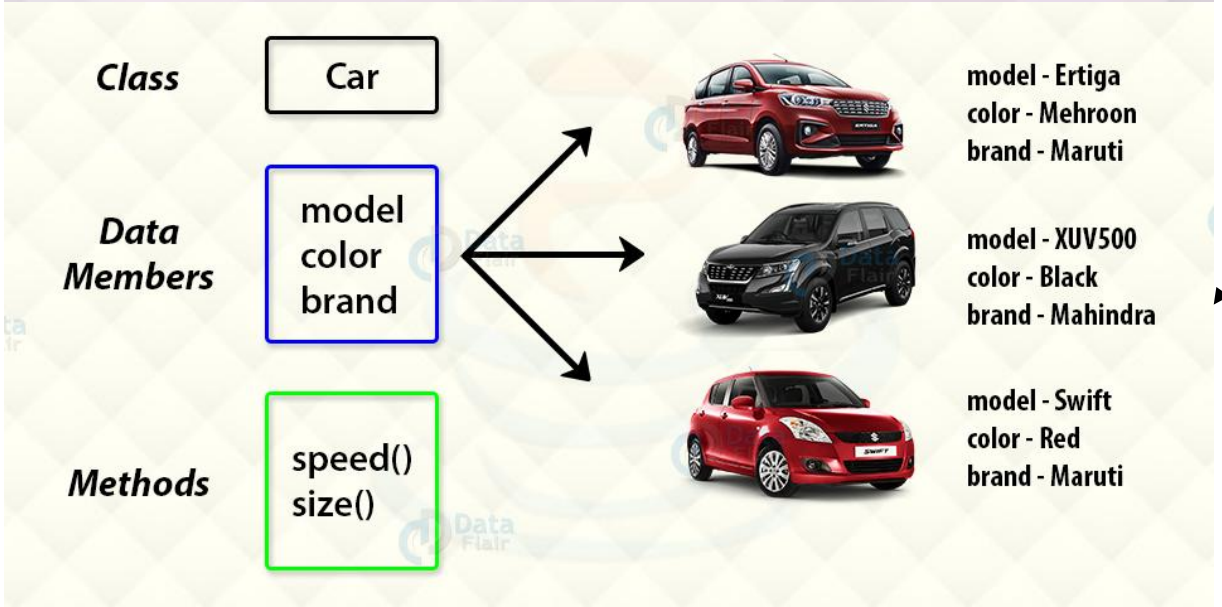
- In C, structures define the representation of a compound value, while functions define operations on these data. In C++, these two ideas are integrated in the forms of class or C++ structure.
- Object-oriented languages are characterized by representing most data structures as ***objects*** that encapsulate both the representation and the behavior of the objects (i.e., operations on the data) in a single entity.
- The C++ object model is based on the idea of a ***class***, which is a template describing all objects of a particular type. The class definition specifies the representation of the object by naming its ***fields*** or ***instance variables*** (e.g., compound property/attribute values) and the behavior of the object by providing a set of ***methods***.
- New objects are created as ***instances*** of a particular class, and the creation of new instances is called ***instantiation***.

# Class and Objects




**Class**

**Objects**



# Representing Points Using a Class

- Declaring public instance variables, however, is discouraged in modern object-oriented programming. Today, the common practice is to make all instance variables **`private`**, which means that clients have no direct access to the internal variables.
- Clients instead use methods exported by the class to obtain access to any information the class contains. Keeping implementation details away from the client is likely to foster *simplicity*, *flexibility*, and *security*, as described in the previous lectures.

```
class Point {
public:
    prototypes of public methods to access x and y
private:
    int x;
    int y;
};
```

# The Format of a Class Definition

- In C++, the definition of a class typically looks like this:

```
class typename {
public:
    prototypes of public methods

private:
    declarations of private instance variables
    prototypes of private methods
};
```

- The entries in a class definition are divided into three categories:
  - A public section available to clients of the class
  - A private section restricted to the implementation
  - A protected section available to derived class.

# Implementing Methods

- A class definition usually appears as a **`.h`** file that defines the ***interface*** for that class. The class definition does not specify the implementation of the methods exported by the class; only the prototypes appear.
- Before you can compile and execute a program that contains class definitions, you must provide the implementation for each of its methods. Although methods can be implemented within the class definition, it is stylistically preferable to define a separate **`.cpp`** file that hides those details. In some rare cases, however, some simple implementations can be included in the **`.h`** file, therefore a **`.cpp`** file is not always required.
- Method definitions are written in exactly the same form as traditional function definitions. The only difference is that you write the name of the class before the name of the method, separated by a double colon. E.g., the **`Point`** class exports a **`toString`** method, you would code the implementation using the method name **`Point::toString`**. ( ≈ **`__str__`** in Python)

# Getters and Setters

- In computer science, methods that retrieve the values of instance variables are formally called ***accessors***, but are more often known as ***getters***. By convention, the name of a getter method begins with the prefix **`get`** followed by the name of the field after capitalizing the first letter in its name. The getters for the **`Point`** class can therefore be defined as **`getX`** and **`getY`**.
- Methods that set the values of specific instance variables are called ***mutators*** or, more informally, ***setters***. If the **`Point`** class were to export a **`setX`** and a **`setY`** method that allowed the client to change the values of these fields, you could easily replace any application that previously used the old structure type with a version that relies entirely on the new **`Point`** class.
- Remember the *getters* and *setters* in the collection classes?

# Setters or not?

- It is, however, counter-intuitive to add setter methods to a class after deciding to make its instance variables private, because part of the reason for making them private is to ensure that clients don't have unrestricted access to them.
- In general, it is considerably safer to allow clients to *read* the values of the instance variables than to *write* those values. As a result, ***setter*** methods are far less common than ***getters*** in object-oriented design.
- Classes can be designed in an even higher level of security by making it impossible to change the values of any instance variables after an object has been created (i.e., ***immutable***).

# Constructors

- In addition to method prototypes, class definitions typically include one or more ***constructors***, which are used to initialize an object.
- The prototype for a constructor has no return type and always has the same name as the class. It may or may not take arguments, and a single class can have multiple constructors (i.e., overloading) as long as the constructors have different signatures.
- The constructor that takes no arguments is called the ***default constructor***. If you don't define any constructors, C++ will automatically generate a default constructor with an empty body.
- The constructor for a class is *always* called when you create an instance of that class, even if you simply declare a variable. The version being called is determined by the arguments.

# Copying Objects

- There is one remaining issue about creating new classes that is extremely important in practice, which is how such objects behave if you copy them.
- When you are defining a new abstract data type in C++, you typically need to define two methods to ensure that copies are handled correctly:
  - The operator **`operator=`**, which takes care of assignments
  - A copy constructor, which takes care of by-value parameters
- These methods have well-defined signatures and structures, and the easiest thing to do is simply to copy the sample code on the next few slides, adapting it as necessary to account for the specific instance variables that need to be copied in the underlying representation.

# Copying Objects

- The process of copying an object is controlled by two standard methods, each of which has a specific prototype.
- The ***assignment operator*** has the following form:

*Return by reference*

*Call by reference*

```
type & type::operator=(const type & src)
Point& Point::operator=(const Point& src)
```

- The prototype for the ***copy constructor*** looks like this:

*Constructors have the same name as the class (type).*

```
type::type(const type & src)
Point::Point(const Point& src)
```

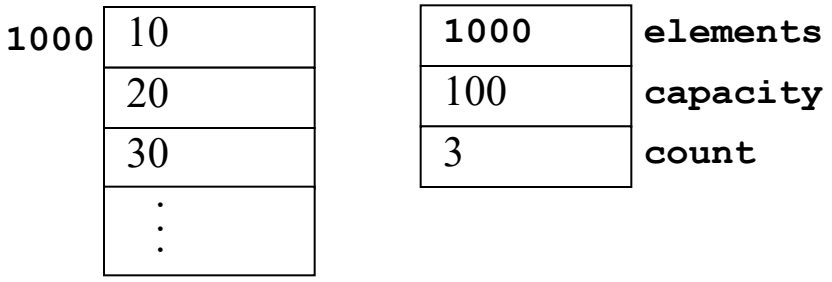
- Call/return by value might cause unnecessary calls to the copy constructor. Constant call by reference protects the source.
- An assignment operator can return anything it wants, but the standard C and C++ assignment operators return a reference to the left-hand operand (e.g., for primitive types). This allows you to chain assignments together. Unless you have a really good reason, you want to follow this convention.

# Assignment and Copy Constructors

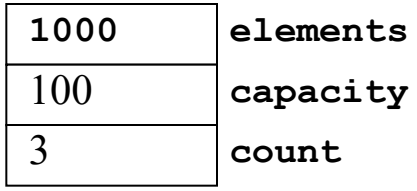
- If you don't provide assignment operator and copy constructor yourself, C++ provides them for you.
- Unfortunately, the default behavior of C++ is to copy only the top-level fields in an object, which means that all dynamically allocated memory is shared between the original and the copy.
- This default behavior, which is called ***shallow copying***, violates the semantics of data structures such as the collection classes. The collection classes in C++ are defined so that copying one collection to another creates an entirely new copy of the collection. What you need to implement instead is ***deep copying***, which copies the data in the dynamically allocated memory as well.
- The simplest strategy for ensuring that clients don't violate the integrity of data structures through assignment is to prevent the client from copying an object altogether by making the assignment operator and copy constructor private methods.

# Shallow *vs.* Deep Copying

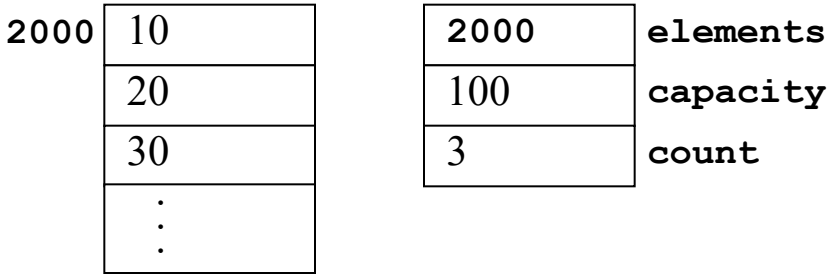
- Suppose that you have an existing **`Vector<int>`** with three elements as shown in the diagram to the right.




- A ***shallow copy*** allocates new fields for the object itself and copies the information from the original. Unfortunately, the dynamic array is copied as an address, not the data.




- A ***deep copy*** also copies the contents of the dynamic array and therefore creates two independent structures

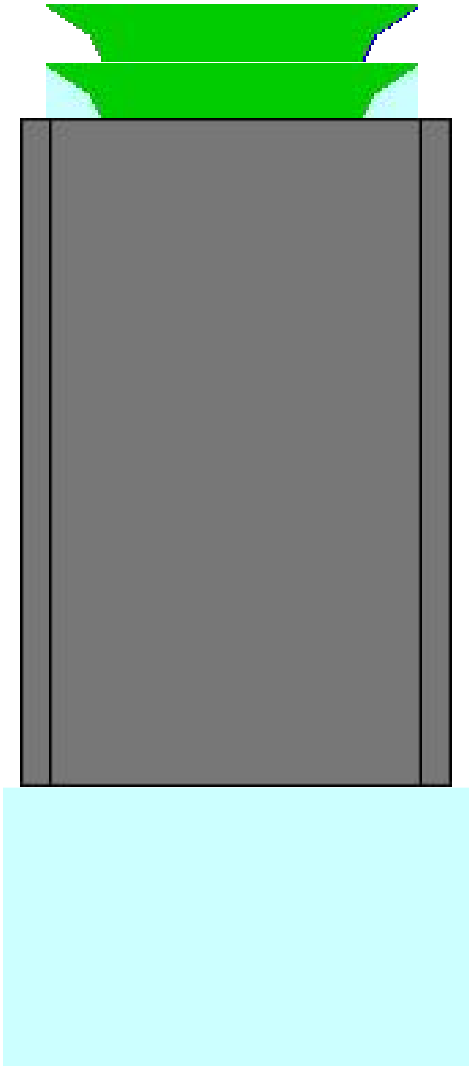


# Implementing Deep Copy Semantics

- This becomes an issue especially when you implement a class that uses dynamic memory.
- The crux of the problem is that copying an abstract data object typically needs to copy the underlying data (***deep copying***) and not just the fields directly accessible in the object.
- Unfortunately, the default interpretation in C++ is to copy only the top-level fields (***shallow copying***), which can lead to serious errors if you are expecting deep copying.
- Therefore, it is important to specify how the object can be properly copied in your implementation.

# The Stack Metaphor

- A ***stack*** is a data structure in which the elements are accessible only in a ***Last In, First Out (LIFO)*** order.
- The fundamental operations on a stack are `push`, which adds a new value to the top of the stack, and `pop`, which removes and returns the top value.
- One of the most common metaphors for the stack concept is a spring-loaded storage tray for dishes. Adding a new dish to the stack pushes any previous dishes downward. Taking the top dish away allows the dishes to pop back up.
- In programming, nested function calls behave in a stack-oriented fashion.



# Code to Implement Deep Copying

```
class CharStack {
public:
   CharStack();
   CharStack(const CharStack & src)
   push(char c);
   char pop();
   CharStack& operator=(const CharStack & src) ;
   ~CharStack();
   int getCount():

priviate:
// Private method prototypes
   void deepCopy(const charStack & src);
   void expandCapacity();
// private data member
   char *array;
   int capacity;
   int count;
}
```

# Code to Implement Deep Copying

```
CharStack::CharStack(const CharStack & src) {
   deepCopy(src);
}

CharStack & CharStack::operator=(const CharStack & src) {
   if (this != &src) {
      delete[] array;
      deepCopy(src);
   }
   return *this;
}

void CharStack::deepCopy(const CharStack & src) {
   if(array) delete [] array; //clear the existing memory space
   array = new char[src.capacity];
   for (int i = 0; i < src.count; i++) {
      array[i] = src.array[i]; //copy one by one
   }
   count = src.count;
   capacity = src.capacity;
}

CharStack::~CharStack () {
   if(array) delete [] array;
}
```

# Initializer List

- An ***initializer list*** is sometimes used in initializing the data members of a class. The initializer list appears just before the brace that begins the body of the constructor and is set off from the parameter list by a colon. The elements of the initializer list should be in one of the following forms:
  - The name of a field in the class, followed by an initializer for that field enclosed in parentheses.

```
Point(int xx, int yy) : x(xx), y(yy) {};
```

  - A superclass constructor (will be discussed later with ***inheritance***).

```
Sqaure(int xx) : Rectangle(xx, xx), x(xx) {};
```

# Initializer List

- The following example of initialization by assignments

```
Point(int xc = 0, int yc = 0) {
   x = xc;
   y = yc;
}
```

  can be optimized for efficiency by using the initializer list:

```
Point(int xc = 0, int yc = 0) : x(xc), y(yc) {}
```

- Using assignments, the members are initialized with default values, and then reassigned in the constructor body.
- Using the initializer list, the members are created and initialized only once, with the given value.
- Constant members can be initialized/re-initialized by using the initializer list, not via assignment because assigning a value to a constant member is not allowed.

# The `point.h` Interface

```
class Point
{
public:
        Point(int xx = 0, int yy = 0) : x(xx), y(yy) {};
        Point(const Point &src);

        int getX() const { return x; }
        int getY() const { return y; }
        void setX(int xx) { x = xx; }
        void setY(int yy) { y = yy; }

        Point &operator+(const Point &pt);
        Point &operator-(const Point &pt);
        Point &operator=(const Point &pt);
        Point operator++();
        Point operator++(int);
        Point operator--();
        Point operator--(int);
        explicit operator bool() const { return x != 0 && y != 0; };
        double length(const Point &p);
        double length();
        ~Point();
        friend std::ostream &operator<<(std::ostream &os, Point &p);

private:
        int x, y;
        std::string toString();
};
```

**Constructors**

**Getters & Setters**

**Operators**

**Boolean conversion**

**Public methods**

**Destructor**

**Friend functions**

**Private data**

**Private method**

# The `point.cpp` Implementation

```
#include <iostream>
#include <sstream>
#include <math.h>
#include <string>
#include "Point.h"

using namespace std;

Point::Point(const Point &src) // copy constructor
{
	this->x = src.x;
	this->y = src.y;
}

Point &Point::operator+(const Point &pt)
{
	this->x += pt.x;
	this->y += pt.y;
	return *this;
}

Point &Point::operator-(const Point &pt)
{
	this->x -= pt.x;
	this->y -= pt.y;
	return *this;
}
```

# The `point.cpp` Implementation

```
Point Point::operator++() // pre-incremental
{
	this->x += 1;
	this->y += 1;
	return *this;
}

Point Point::operator++(int) // post-incremental
{
	Point tmp = *this;
	this->x += 1;
	this->y += 1;
	return tmp;
}

Point Point::operator--() // pre-decremental
{
	this->x -= 1;
	this->y -= 1;
	return *this;
}

Point Point::operator--(int) // post-decremental
{
	Point tmp = *this;
	this->x -= 1;
	this->y -= 1;
	return tmp;
}
```

# The `point.cpp` Implementation

```
Point &Point::operator=(const Point &pt)
{
	this->x = pt.x;
	this->y = pt.y;
	return *this;
}

double Point::length(const Point &p)
{
	double dx = p.x - this->x;
	double dy = p.y - this->y;
	return sqrt(dx * dx + dy * dy);
}

double Point::length()
{
	double dx = this->x;
	double dy = this->y;
	return sqrt(dx * dx + dy * dy);
}
```

# The `point.cpp` Implementation

```
string Point::toString()
{
	ostringstream os;
	os << ''[" << this->x << "," << this->y << ''] length =" << this->length();
	return os.str();
}

Point::~Point()
{
}

ostream &operator<<(ostream &os, Point &p)
{
	os << p.toString() << endl;
	return os;
}
```

# Overloading Operators

- Besides ***methods***, you can also implement some useful ***operators***, even those existing ones in C++.
- One of the most powerful features of C++ is the ability to extend the existing operators so that they apply to new types. Each operator is associated with a name that usually consists of the keyword **`operator`** followed by the operator symbol.
- For instance, one of the most useful operators to overload is the insertion operator **<<**, which makes it easy to display values of that class on an output stream.
- We can use the following line:

```
cout << "[" << this->x << "," << this->y << "]:length =" << this->length();
```

or define a string representation of the object:

```
cout << pt.tostring();
```

but it would be better if we could just:

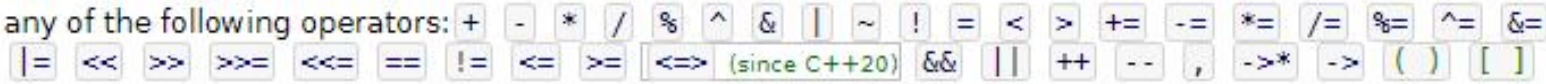
```
cout << pt;
```

# Overloading Operators

- The prototype and implementation for the overloaded **<<** in the **`Point`** class is:

```
ostream & operator<<(ostream & os, Point pt) {
    os << pt.toString();
    return os;
}
```

- Since stream variables cannot be copied, the **`ostream`** argument must be passed by reference.
- The **<<** operator has a chaining behavior of returning the output stream, therefore, the definition of **`operator<<`** must also return its result by reference.
- Other operators that can be overloaded:



# Overloading Operators

- To overload the == operator, i.e., to allow statements like this:

```
if (p1 == p2)
```

*Or we can treat this operator as a free function as if:* `operator==(p1,p2)`

- The method-based style:

```
// in point.h, inside the class Point definition
bool operator==(Point pt);
// in point.cpp
bool Point::operator==(Point pt) {
   return x == pt.x && y == pt.y;
}
```

`p1` *is the receiver and the argument* `p2` *is copied to parameter* `pt`*. The private data of a class are private to the class and not to the object.*

- The free function approach:

```
// in point.h inside the class Point definition,
friend bool operator==(Point p1, Point p2);

// in point.cpp
bool operator==(Point p1, Point p2) {
   return p1.x == p2.x && p1.y == p2.y;
}
```

*Friend function can access the provide data member*

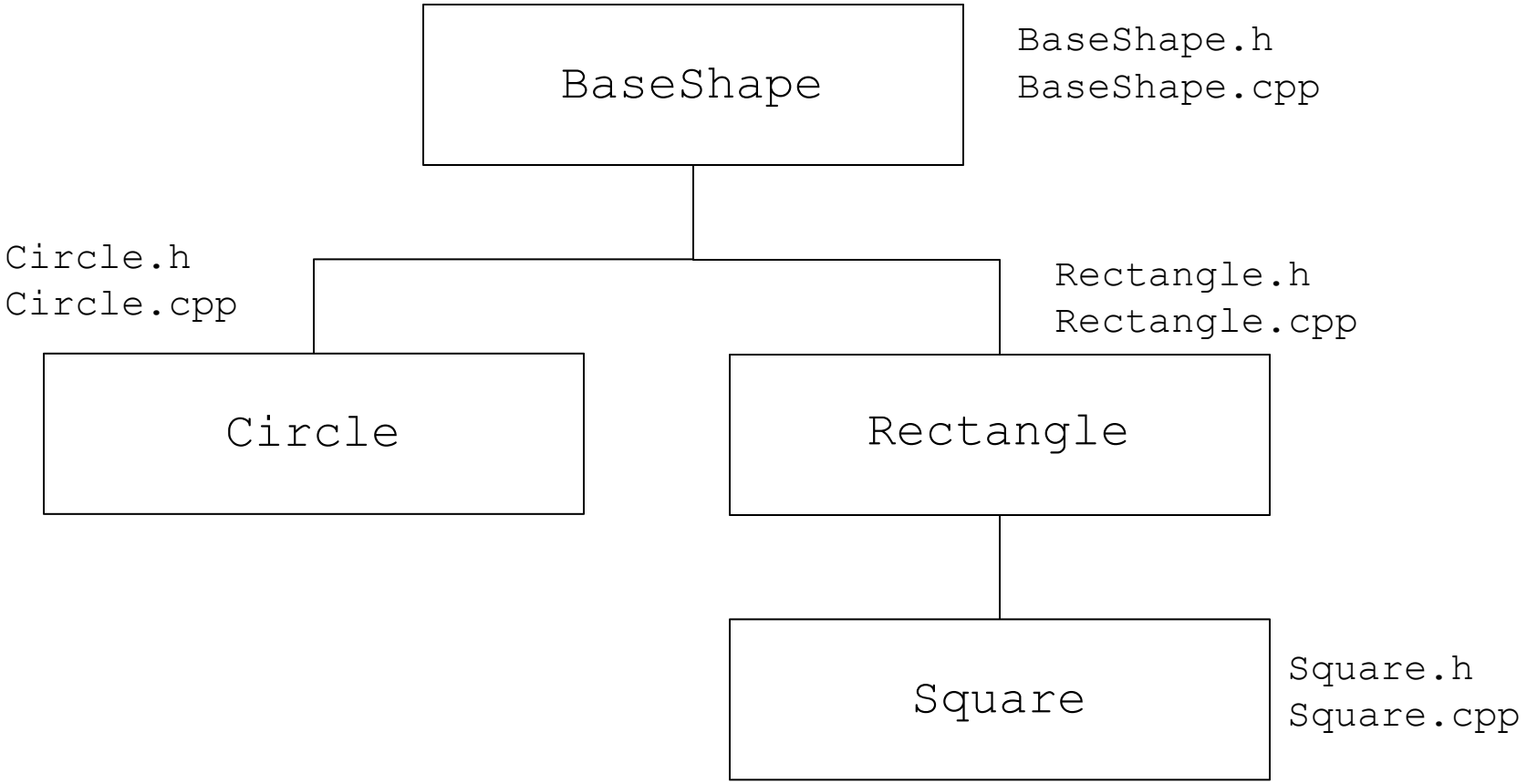
# Overloading Operators

- When you define operators for a class, you can write them either as class methods (object-oriented programming style) or as free functions (procedural programming style). Each style has its own advantages and disadvantages.
  - Defining an operator as a class method means that the operator is part of the class and therefore has free access to the private instance variables and other methods. But when you use this style to overload a binary operator, the left operand is the receiver object and the right operand is passed as a parameter.
  - Defining an operator as a free function often produces code that is easier to read (the operands for a binary operator are both passed as parameters as usual), but also means that the operator function must be designated as a `friend` to access to the private data.

# A Strategy for Defining New Classes

- Think generally about how clients are likely to use the class.
- Determine what information belongs in the private state of each object.
- Define a set of constructors to create new objects.
- Enumerate the operations (including the overloaded functions and operators) that will become the public methods of the class.
- If necessary, define free functions that can manipulate the objects outside the class definition, but make them friends to the class.
- Code (as the implementer) and test (as the client) the implementation.
- Keeping implementation details away from the client is likely to foster *simplicity*, *flexibility*, and *security*, as described in the previous lectures (compare this to libraries).

# Virtual Functions and Abstract Classes



# Virtual Functions and Abstract Classes

- Once a method is declared as a virtual method, it becomes virtual in every subclass. In other words, it is not necessary to use the keyword virtual in the subclasses.
- Any method that must be implemented in the concrete subclasses is indicated by adding **= 0** before the semicolon on the prototype line, to mark the definition of a ***pure virtual method***.

```
virtual double calcArea() = 0;
```

- Any class that has pure virtual method is call abstract class.
- You cannot declare an object of an abstract class because some of the methods may not be properly implemented (e.g., pure virtual methods). But you can declare a pointer of an abstract class, which can be later used to point to an object of a concrete subclass inheriting from the abstract superclass.

```
BaseShape bs;
```



```
BaseShape* pBS;
```






# Example: Rational Numbers

- As a more elaborate example of class definition, section 6.3 defines a class called **`Rational`** that represents ***rational numbers***, which are simply the quotient of two integers.
- Rational numbers can be useful in cases in which you need exact calculation with fractions (e.g., $1/3 = 0.\dot{3}$).
- Even if you think $1/10 = 0.1$ is exact, when you use a **`double`**, 0.1 is represented internally as an approximation .
- Rational numbers support the standard arithmetic operations:

Addition:

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd}$$

Subtraction:

$$\frac{a}{b} - \frac{c}{d} = \frac{ad - bc}{bd}$$

Multiplication:

$$\frac{a}{b} \times \frac{c}{d} = \frac{ac}{bd}$$

Division:

$$\frac{a}{b} \div \frac{c}{d} = \frac{ad}{bc}$$




# Implementing the `Rational` Class

- The private instance variables: ***numerator*** and ***denominator***.
- The ***constructors*** for the class are overloaded. Calling the constructor with no argument creates a `Rational` initialized to 0, calling it with one argument creates a `Rational` equal to that integer, and calling it with two arguments creates a fraction. The following properties should be enforced: (why?)
  - The fraction is always expressed in lowest term.
  - The denominator is always positive.
  - The rational number 0 is always represented as the fraction 0/1.

  Since these values never change once a new `Rational` is created, these properties will remain in force.
- The class overloads the standard arithmetic operations ( `+` , `-` , `*` , `/` ) to allow the use of conventional mathematical notation.

# The `rational.h` Interface

```
/*
 * File: rational.h
 * ----------------
 * This interface exports a class representing rational numbers.
 */

#ifndef _rational_h
#define _rational_h

#include <string>
#include <iostream>

/*
 * Class: Rational
 * ---------------
 * The Rational class is used to represent rational numbers, which
 * are defined to be the quotient of two integers.
 */

class Rational {
```

# The `rational.h` Interface

```
public:

/*
 * Constructor: Rational
 * Usage: Rational zero;
 *        Rational num(n);
 *        Rational r(x, y);
 * ------------------------
 * Creates a Rational object.  The default constructor creates the
 * rational number 0.  The single-argument form creates a rational
 * equal to the specified integer, and the two-argument form
 * creates a rational number corresponding to the fraction x/y.
 */

   Rational();
   Rational(int n);
   Rational(int x, int y);
```

# The `rational.h` Interface

```
/*
 * Operators: +, -, *, /
 * ---------------------
 * Define the arithmetic operators.
 */

   Rational operator+(Rational r2);
   Rational operator-(Rational r2);
   Rational operator*(Rational r2);
   Rational operator/(Rational r2);

/*
 * Method: toString()
 * Usage: string str = r.toString();
 * ---------------------------------
 * Returns the string representation of this rational number.
 */

   std::string toString();
```

# The `rational.h` Interface

```
private:

/* Instance variables */

   int num;     /* The numerator of this Rational object   */
   int den;     /* The denominator of this Rational object */

};

/*
 * Operator: <<
 * Usage: cout << rat;
 * -------------------
 * Overloads the << operator so that it is able to display
 * Rational values.
 */

std::ostream & operator<<(std::ostream & os, Rational rat);

#endif
```

# The `rational.cpp` Implementation

```
/*
 * File: rational.cpp
 * ------------------
 * This file implements the Rational class.
 */

#include <string>
#include <cstdlib>
#include "rational.h"
#include "strlib.h"
using namespace std;

/* Function prototypes */

int gcd(int x, int y);

/* Constructors */
```

*Why is the prototype of* **`gcd`** *not in* **`rational.h`** *or the* **`Rational`** *class definition?*

# The `rational.cpp` Implementation

```
Rational::Rational() {
   num = 0;
   den = 1;
}

Rational::Rational(int n) {
   num = n;
   den = 1;
}

Rational::Rational(int x, int y) {
   if (x == 0) {
      num = 0;
      den = 1;
   } else {
      int g = gcd(abs(x), abs(y));
      num = x / g;
      den = abs(y) / g;
      if (y < 0) num = -num;
   }
}
```

# The `rational.cpp` Implementation

```
/* Implementation of the arithmetic operators */

Rational Rational::operator+(Rational r2) {
   return Rational(num * r2.den + r2.num * den, den * r2.den);
}

Rational Rational::operator-(Rational r2) {
   return Rational(num * r2.den - r2.num * den, den * r2.den);
}

Rational Rational::operator*(Rational r2) {
   return Rational(num * r2.num, den * r2.den);
}

Rational Rational::operator/(Rational r2) {
   return Rational(num * r2.den, den * r2.num);
}
```

*Overloading as member functions*

# The `rational.cpp` Implementation

```
string Rational::toString() {
   if (den == 1) {
      return integerToString(num);
   } else {
      return integerToString(num) + "/" + integerToString(den);
   }
}

int gcd(int x, int y) {
   int r = x % y;
   while (r != 0) {
      x = y;
      y = r;
      r = x % y;
   }
   return y;
}

ostream & operator<<(ostream & os, Rational rat) {
   os << rat.toString();
   return os;
}
```

# Exercise: overloading as free functions?

```
// in rational.h, inside the class Rational definition
friend Rational operator+(Rational r1, Rational r2);
friend Rational operator-(Rational r1, Rational r2);
friend Rational operator*(Rational r1, Rational r2);
friend Rational operator/(Rational r1, Rational r2);

// in rational.cpp
Rational operator+(Rational r1, Rational r2) {
   return Rational(r1.num * r2.den + r2.num * r1.den, r1.den * r2.den);
}
Rational operator-(Rational r1, Rational r2) {
   return Rational(r1.num * r2.den - r2.num * r1.den, r1.den * r2.den);
}
Rational operator*(Rational r1, Rational r2) {
   return Rational(r1.num * r2.num, r1.den * r2.den);
}
Rational operator/(Rational r1, Rational r2) {
   return Rational(r1.num * r2.den, r1.den * r2.num);
}
```




# Designing a token scanner class

- Many applications need to divide a string into words, or more generally, into logical units that may be larger than a single character. In computer science, such units are typically called ***tokens***.
- Different applications, however, define tokens in all sorts of different ways, which means that the **`TokenScanner`** class must give the client some control over what types of tokens are recognized. E.g., the following line of text can be scanned into many different possible sequences of tokens.

```
cout << "hello, world" << endl;
```

- The Stanford C++ library includes a **`TokenScanner`** class that offers considerable flexibility without sacrificing simplicity.

# Designing a token scanner class

- From the client's point of view:

```
Set the input for the token scanner to be some string or input stream.
while (more tokens are available) {
    Read the next token.
    Perform other operations using the token.
}
```

- From the implementer's point of view:
  - A **`setInput`** method that allows clients to specify the token source, ideally, overloaded to take either a string or an input stream.
  - A **`hasMoreTokens`** method that tests whether the token scanner has any tokens left to process.
  - A **`nextToken`** method that scans and returns the next token.

# Methods in the `TokenScanner` Class

| |
|---|
| **`scanner.setInput(str)`** *or* **`scanner.setInput(infile)`**<br>Sets the input for this scanner to the specified string or input stream. |
| **`scanner.hasMoreTokens()`**<br>Returns **`true`** if more tokens exist, and **`false`** at the end of the token stream. |
| **`scanner.nextToken()`**<br>Returns the next token from the token stream, and `""` at the end. |
| **`scanner.saveToken(token)`**<br>Saves **`token`** so that it will be read again on the next call to **`nextToken`**. |
| **`scanner.ignoreWhitespace()`**<br>Tells the scanner to ignore whitespace characters. |
| **`scanner.scanNumbers()`**<br>Tells the scanner to treat numbers as single tokens. |
| **`scanner.scanStrings()`**<br>Tells the scanner to treat quoted strings as single tokens. |



# Encapsulating Programs as Classes

- The primary advantage of doing so is that classes provide better encapsulation. The fact that access to any private data is limited to the class itself means that it is much safer to use private instance variables to share information than it is to use global variables, which offer no such security.
- The information shared between these functions can now be stored in instance variables and need not be passed as arguments. Being able to share access to such data among all the methods in the class substantially reduces the size and complexity of the parameter lists.

# Encapsulation

Encapsulation is a fundamental concept of object-oriented programming (OOP) that involves bundling data (attributes) and methods (functions) that operate on the data into a single unit called a class. It restricts direct access to some of an object's components, which is a means of preventing unintended interference and misuse of the data

# Polymorphism

Polymorphism in C++ is a core concept of object-oriented programming that allows objects of different classes to be treated as objects of a common superclass. It enables a single interface to represent different underlying forms (data types). There are two main types of polymorphism in C++: compile-time polymorphism (overloading) and runtime polymorphism (overriding).

1. Compile-Time Polymorphism (Function Overloading)

Function overloading allows multiple functions to have the same name but different parameter lists (number, type, or order of parameters). The compiler determines which function to call based on the arguments provided.

2. Runtime Polymorphism (Function Overriding)

Runtime polymorphism is achieved through inheritance and virtual functions. It allows a subclass to override a method of its superclass. The actual method to be called is determined at

# Inheritance

Inheritance in C++ is a fundamental concept of object-oriented programming (OOP) that allows a new class (called the derived or child class) to inherit properties and behaviors (i.e., data members and member functions) from an existing class (called the base or parent class). It promotes code reusability and establishes a hierarchical relationship between classes.

**Si**
```
class Base {};
class Derived : public Base {};
```
A derived class inherits from a single base class.

**M**
```
class Base {};
class Derived1 : public Base {};
class Derived2 : public Derived1 {};
```
: A class inherits from a derived class.

**M**
```
class Base1 {};
class Base2 {};
class Derived : public Base1, public Base2 {};
```
inherits from multiple base classes

**H**
```
class Base {};
class Derived1 : public Base {};
class Derived2 : public Base {};
```
**nce**: Multiple derived classes inherit from a single base class.

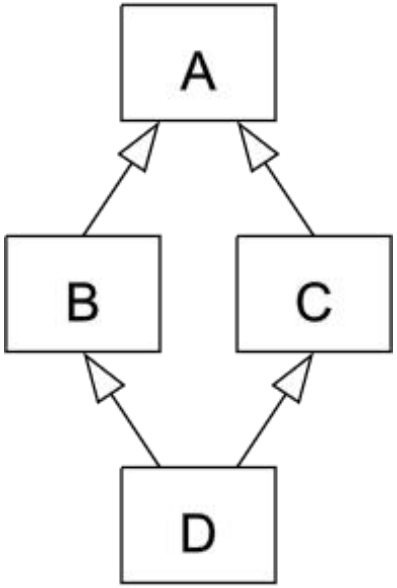
# Inheritance

- Subclasses have access to the public and protected (but not private) members/methods in the superclass.
- ***public***, ***protected***, and ***private*** inheritance have the following features:

| Base class member access specifier | Type of Inheritence | | |
|---|---|---|---|
| | Public | Protected | Private |
| Public | Public | Protected | Private |
| Protected | Protected | Protected | Private |
| Private | Not accessible (Hidden) | Not accessible (Hidden) | Not accessible (Hidden) |

# Multiple Inheritance

- Although multiple inheritance has its uses, it tends to create more problems than it solves.
- The "deadly diamond of death" is an ambiguity that arises when two classes B and C inherit from A, and class D inherits from both B and C. If there is a method in A that B or C has overridden method, and D does not override it, then which version of the method does D inherit: that of B, or that of C?
- It is an ambiguity to access the data member of class A as well in this case.



# Virtual Inheritance

- Different languages have different ways of dealing with these problems of multiple inheritance. C++ by default follows each inheritance path separately, so a D object would actually contain two separate A objects, and uses of A's members have to be properly qualified.
- If the inheritance from A to B and the inheritance from A to C are both marked `virtual`, C++ takes special care to only create one A object, and uses of A's members work correctly.

# Virtual Inheritance

```
class A
{
      public:
      int a = 1;
      void virtual display() { cout << "a=" << a << endl; }
};

class B : virtual public A
      {
      public:
      int b = 2;
      void display() { cout << "a=" << a << " b=" << b << endl; }
};

class C : virtual public A
{
      public:
      int c = 3;
      // void display() { cout << "a=" << a << " c=" << c << endl; }
};

class D : public B, public C {};

int main()
{
      D oD;
      oD.a = 100;
      oD.display();
}
```

**Virtual inheritance**

**Virtual inheritance**

→ **a=100 b=2**

The End